

Twelve methods of the incident pipeline

From files on disk to a report you can defend.

A plain-language walk through the twelve architecture methods in this repository: what each stage does, why it exists, where it lives in the code, and which AI ideas sit next to it even when this project does not implement them.

Default path: deterministic ingest, detect, correlate, RCA, heuristic compose.

Optional AI: OpenAI gpt-4.1-mini as a grounded report rewriter.

Not in this app: learned anomaly ML, learned RCA.

How to read this

This project is a full-stack incident analysis platform: Next.js console, FastAPI, and a Python compiler for logs and metrics. Most of the pipeline is classical software (schemas, statistics, a dependency graph). An optional large language model rewrites the report after evidence is ranked. That design is the point: detection does not wait on a model, and every claim can be cited.

The twelve methods below are the high-level architecture flow. They are not the same numbering as the eight runtime stages on the Method page (Ingest through Review). Those eight are the operator view. These twelve are the engineering view, including prompts, retrieval, grounding checks, artifacts, and delivery.

THE PATH IN ONE LINE

Logs + metrics -> ingest -> normalize -> detect -> correlate -> RCA -> grounded context -> prompts -> retrieve runbooks -> compose (heuristic or OpenAI) -> validate claims -> persist artifacts -> CLI / API / console / webhook.

THREE KINDS OF "AI"

1. Rules and statistics (this project, heavily): z-score, MAD, weighted RCA. A human wrote the rule.
2. Machine learning (not in this app): train on labeled incidents. Isolation Forest, neural nets, learned RCA.
3. Generative LLMs (optional): gpt-4.1-mini rewrites prose from the evidence bundle. The website does not call OpenAI; the API does, if you turn it on.

01 Ingestion

Load logs and metrics into typed records

WHAT IT DOES

Ingestion is the front door. Files of logs and metrics enter as CSV, JSON, or JSONL. Optional Prometheus query_range can supply metrics instead of a file. Each row is validated against a schema: LogEvent (timestamp, service, severity, message) or MetricPoint (timestamp, service, metric name, value).

WHY IT EXISTS

An AI system that cannot trust its inputs will invent them. Invalid rows are counted in a data-quality report instead of crashing the run or disappearing. That is how operators know the file was dirty.

IN THIS REPOSITORY

Code lives in src/incident_agent/ingestion/. CLI: incident-agent ingest-data and validate-data. Console runs pass bundled sample paths such as data/sample/incident/anomaly_logs.csv.

CONCEPTS TO LEARN NEXT

Schema (a typed contract both sides agree on). Data quality. ETL (Extract, Transform, Load). In production this layer would also talk to Kafka, CloudWatch, Datadog, or Splunk.

02 Normalization

One clock. One timeline. Shared buckets.

WHAT IT DOES

Clocks lie. One file is UTC, another has no timezone. Everything is converted to UTC. Events then fold into time buckets so detectors share one clock. The default bucket is five minutes. Missing data has a policy: traffic missing can be zero; CPU missing stays unknown; a lost heartbeat can mean unavailability.

WHY IT EXISTS

A latency spike and an error-log burst can only be correlated if they land in the same window. Without alignment, the rest of the pipeline compares apples to oranges.

IN THIS REPOSITORY

src/incident_agent/normalization/timeline.py. Config: `normalization.bucket_size_minutes` in `configs/default.yaml` (1, 5, or 15). Output: `normalized/timeline.json`.

CONCEPTS TO LEARN NEXT

Time series. Feature windows. Missing-data semantics. This is the unglamorous work that every metrics ML project also needs before a model sees a number.

03 Anomaly detection

Statistical gates, not a trained model

WHAT IT DOES

An anomaly is a number that does not look like its recent self. Eight independent families fire: error rate, latency, CPU, memory, traffic drop, unavailability, ERROR log burst, CRITICAL log burst. Each uses a rolling median baseline, MAD (median absolute deviation) for spread, a z-score-style threshold, minimum relative change, and minimum support (enough points).

WHY IT EXISTS

Quiet CPU must not hide a screaming error rate. Independent detectors first, merge later. Observed vs baseline is stored on every hit so a report cannot say "slightly elevated" when the file says 1900ms against 125ms.

IN THIS REPOSITORY

src/incident_agent/anomaly_detection/. Thresholds in configs/default.yaml under anomaly_detection. Output: AnomalyCandidate rows in anomalies/anomalies.json. This is NOT machine learning. No sklearn, no neural net.

CONCEPTS TO LEARN NEXT

Mean vs median. Standard deviation vs MAD. Z-score: how many spreads above typical. False positives vs false negatives. Beyond this repo: Isolation Forest, seasonality (traffic always high at 9am), change-point detection, multivariate detectors.

04 Correlation

Fourteen alerts become one incident

WHAT IT DOES

Correlation here means grouping, not the statistics formula r . Anomalies join when they share a time window, a service, a dependency-graph edge, or several signals on the same owner. Same-family evidence is intentionally weak so two lonely latency spikes cannot cluster by themselves. A relationship threshold of 1.0 keeps isolated noise out of the incident list.

WHY IT EXISTS

On-call does not want fourteen dashboards. They want one candidate: these signals belong to the same failure.

IN THIS REPOSITORY

src/incident_agent/correlation/. Weights in configs/default.yaml (temporal 0.4, same_service 1.0, dependency 0.8, cross_signal 0.5, same_family 0.2). Graph: configs/service_dependencies.yaml. Output: CorrelatedIncidentCandidate.

CONCEPTS TO LEARN NEXT

Topology / service maps. Blast radius. In industry, traces (OpenTelemetry) make this stronger than logs plus metrics. Graph neural nets exist in research; this repo uses a hand-written YAML graph you can read.

05 RCA - root-cause analysis

Who started the fire, not who is screaming

WHAT IT DOES

RCA ranks origin. Downstream services hurt (the hallway alarm). The cause may be upstream (the kitchen). This project scores each service by detector severity, adds bonuses for failure-like signals, uses the graph so downstream pain is visible, and prefers origin when the topology agrees. Root-cause support equals top score divided by the sum of all candidate scores. If the top two are close, both names stay as ambiguities.

WHY IT EXISTS

Restarting every waiter because they are noisy is how you make the outage worse. Contain the origin.

IN THIS REPOSITORY

src/incident_agent/rca/scoring.py. Output: RootCauseHypothesis in rca/rca_hypotheses.json. README: RCA is heuristic and evidence-ranked, not learned. Support is a ranking inside one incident, not a calibrated probability. Do not tell a VP there is a 77 percent chance.

CONCEPTS TO LEARN NEXT

Correlation is not causation. Five Whys. Fault trees. Causal inference. Learned RCA (train on past incidents) is a different field and is not in this application. Counterfactuals: if we had not deployed, would latency still spike?

06 Grounded context

Only evidence the pipeline already has

WHAT IT DOES

Before any prose is written, the system packs a grounded bundle: ranked evidence, incident features, the RCA hypothesis, optional retrieved snippets. This is the context a model is allowed to see. If checkout CPU was 96, that number is in the bundle. If it is not in the bundle, the model does not get to know it.

WHY IT EXISTS

A large language model has no live Grafana. It predicts the next word. The only honest numbers it can write are numbers you handed it.

IN THIS REPOSITORY

PromptRenderContext requires EvidenceBundle, IncidentSummaryFeatures, RootCauseHypothesis, and optional RetrievedSnippet objects. See `src/incident_agent/prompts/` and `docs/prompting.md`.

CONCEPTS TO LEARN NEXT

Context window. Grounding vs vibes. This is the difference between pasting a CSV into ChatGPT and an AI systems pipeline.

07 Prompt templates

Modular prompts, not a chat box

WHAT IT DOES

Templates render five jobs from the same artifacts: incident summary, root-cause explanation, executive summary, engineering handoff, remediations. Instructions say: use only provided evidence, do not invent services or timestamps, separate facts from inference, include uncertainty when evidence is thin.

WHY IT EXISTS

Free-form paste is not testable. A template given the same EvidenceBundle always builds the same skeleton. You can unit-test prompts without calling OpenAI.

IN THIS REPOSITORY

src/incident_agent/prompts/templates.py and renderer.py. Final shape: FinalIncidentReport in schemas.

CONCEPTS TO LEARN NEXT

Prompt engineering. System vs user messages. Few-shot examples. JSON mode. Tool calling. Chain-of-thought (often you should not show it to users).

08 Retrieval (RAG-lite)

Runbooks and past incidents as citations

WHAT IT DOES

A runbook is a checklist (if checkout latency is high, inspect the database before blaming the edge). Historical incidents are last year's writeups. Markdown is chunked, scored against the current incident by lexical overlap (shared words and service names, not vector embeddings), top_k defaults to 3, snippets clip to 360 characters, each keeps a citation_id.

WHY IT EXISTS

The model should not be the only historian. Procedures belong in files you can diff, not in weights you cannot inspect.

IN THIS REPOSITORY

data/knowledge/runbooks and data/knowledge/incidents. Console jobs set retrieval_enabled. Paths must sit inside the read allowlist (data, configs, artifacts, eval).

CONCEPTS TO LEARN NEXT

RAG: Retrieval-Augmented Generation. Industry RAG usually embeds text into vectors and searches a vector database (Pinecone, Chroma, pgvector). This repo uses a simpler keyword-ish cousin. Same purpose, no extra service. Next study: embeddings, cosine similarity, chunking, rerankers.

09 LLM provider

Heuristic compose by default, OpenAI optional

WHAT IT DOES

The provider interface has two backends. HeuristicNarrativeProvider stitches sentences from evidence JSON. No neural net, no API bill, deterministic. OpenAIProvider calls [https://api.openai.com/v1 chat completions](https://api.openai.com/v1/chat/completions) with gpt-4.1-mini when llm.provider is openai and INCIDENT_AGENT_OPENAI_API_KEY is set. Generic OPENAI_API_KEY is ignored. The Next.js website never calls a model; FastAPI does.

WHY IT EXISTS

The product must run from a clean clone. Generation is the last mile. Detection and RCA do not wait on a model. If the provider fails, incidents and artifacts still exist.

IN THIS REPOSITORY

src/incident_agent/llm/ (base, heuristic, openai_provider, factory, cache). configs/default.yaml: llm.provider heuristic. Optional models: report_model and completion_model gpt-4.1-mini.

CONCEPTS TO LEARN NEXT

LLM: large language model, a next-token predictor. Inference vs training. Hallucination: fluent text that is not true. Provider abstraction: swap backends without rewriting RCA. Agents vs pipelines: this repo is a pipeline with an optional LLM stage, not a chatbot that loops on tools.

10 Grounding validation

If it is not in the bundle, it is unmarked

WHAT IT DOES

After prose exists, each factual sentence is checked against evidence ids, the hypothesis, and retrieved snippets. Statuses: supported (overlap at or above 0.34), unsupported (pretty sentence, no proof), contradictory (fights detectors), not_applicable (remediations are advice, not claims about the past). Policy warn keeps and flags. Policy fail can drop the report.

WHY IT EXISTS

Generation without grounding is a demo. Generation with grounding is a product. Unsupported claims stay visible with a reason. They are not deleted in shame and not promoted to facts.

IN THIS REPOSITORY

grounding/ in config and src/incident_agent/grounding/. Artifact: grounding/grounding_summary.json. Evaluation tracks factual support rate and unexpected service mentions.

CONCEPTS TO LEARN NEXT

Factuality evaluation. Citation attribution. Guardrails. RAG reduces hallucinations; it does not delete them. You still check.

11 Pipeline orchestration and artifacts

A run is a directory, not a chat log

WHAT IT DOES

The orchestrator runs stages in order and writes inspectable JSON: timeline, anomalies, incidents, RCA, grounding, final reports, run_summary. The summary records completed stages, warnings, degraded flags, and token usage. Intermediate and compose caches live under artifacts/cache.

WHY IT EXISTS

If a later stage degrades, earlier JSON remains. Reviewers can disagree with the machine in public. Tomorrow you can re-open the same run.

IN THIS REPOSITORY

src/incident_agent/services/pipeline.py. Console: POST /analysis-jobs. Jobs are in-memory; artifacts persist on disk. Restarting uvicorn clears the board, not the JSON under artifacts/ui/.

CONCEPTS TO LEARN NEXT

Compilers emit IR (intermediate representation). This pipeline treats incidents the same way. Degraded execution: missing logs or metrics become warnings, not a blank page. Idempotency and audit trails.

12 Delivery

CLI, API, console, export, review, webhook

WHAT IT DOES

Humans receive the result through Typer CLI, FastAPI, and the Next.js console. Reports export as JSON, Markdown, and HTML. Review states: draft, reviewed, approved, rejected. Only approved reports may POST to an allowlisted webhook. Every transition stores reviewer, note, and timestamp. Deliveries append JSONL.

WHY IT EXISTS

Human in the loop. The machine proposes. A person stamps. In real operations a report that can leave the building without a name is a liability.

IN THIS REPOSITORY

API on :8000, studio on :3000 with /backend proxy. Path allowlists for reads (data, configs, artifacts, eval) and writes (artifacts only). No end-user auth: trusted workstation, not public SaaS.

CONCEPTS TO LEARN NEXT

REST. Job stores vs durable queues. Webhooks. Approval workflows. Full stack: UI plus API plus engine. Production hardening (auth, multi-tenant isolation) is intentionally out of scope in this version.

How to talk about it

Strong and true: you built a full-stack incident analysis platform with a deterministic detect-correlate-RCA pipeline, retrieval, grounding, and an LLM provider you can switch from heuristic compose to OpenAI.

Also true, next breath: anomaly detection and RCA are statistical and heuristic, not trained models. The LLM is the report layer. That is a design choice so the product works without a key and so every claim can be cited.

SUGGESTED STUDY ORDER IN THE REPO

1. docs/architecture.md - this twelve-step flow.
2. docs/anomaly_detection.md and anomaly_detection/common.py - z-score and MAD.
3. docs/correlation.md and configs/service_dependencies.yaml - the graph.
4. docs/rca.md and rca/scoring.py - support vs probability.
5. llm/heuristic.py vs llm/openai_provider.py - two backends, one interface.
6. A real artifacts/.../grounding_summary.json after running Checkout cascade.
7. web/src/lib/api.ts - what the console actually calls.

HONEST LIMITS

Local files (or optional Prometheus). Hand-written dependency graph. No CloudWatch or Datadog connectors yet. No user login. Jobs live in API memory; JSON lives on disk. Production packaging is demo-oriented. OpenAI is the only real LLM adapter.

The gym is this repository. The next weights are embeddings, Isolation Forest, and causal RCA. You already have the loop from file to evidence to ranked origin to grounded prose to UI.